# Cyber Grid: Classes and For Loops

A clear, source-based study guide for understanding the game code and explaining it in a project defense.

## What this guide covers

This guide explains the three JavaScript classes in Cyber Grid and all 61 explicit **`for`** loops found in its main game script. Each loop entry gives its source line and a plain-language description. Line numbers refer to **`script.js`** in the Cyber Grid project.

The game also uses array methods such as **`forEach`**, **`map`** and **`filter`**. Those can repeat work too, but this guide counts only statements written with the **`for (...)`** syntax. CSS names like **`.class`** and JavaScript **`class`** declarations are different things.

## The three JavaScript classes

### ScoreBoard - script.js:58-66

Stores the X and O win totals. Its constructor prepares the scoreboard by calling reset(). addWin(player) checks that the player is X or O, then adds one point. reset() returns both totals to zero. This class supports match scoring.

### BusLine - script.js:1879-1910

Represents one decorative circuit route. It stores the route points, line width and color, then draws the connected path on a canvas. It controls background presentation, not the tic-tac-toe rules.

### CircuitPulse - script.js:1912-1961

Represents a glowing pulse traveling along a BusLine. update() moves it forward and handles progress along the route; draw() paints the moving pulse. It creates ambient circuit animation.

**How to explain a class:** A class is a reusable blueprint for objects. Here, ScoreBoard models game score behavior, while BusLine and CircuitPulse model two visual objects used by the animated circuit background.

## A useful first-study order

Start with ScoreBoard because it connects directly to winning and score updates. Then read the game-state and win-checking functions. Study BusLine and CircuitPulse last because they create background effects rather than deciding who wins.

# 1. Loops in game rules and setup

A for loop repeats a block of code. The common pattern has a starting value, a condition, and an update. For example, `for (let i = 0; i < 9; i++)` visits indexes 0 through 8, which match the nine board cells.

## Rules, AI, scoring and saved matches

| LINE | WHAT THE LOOP DOES |
|---|---|
| 102 | Checks each possible winning line to see whether X or O has made three in a row. |
| 128 | Classic AI tries each empty board cell as a possible move. |
| 143 | The search function recursively examines the ordered list of possible moves. |
| 152 | Compares candidate first moves so the AI can select its best result. |
| 732 | Updates the visible score for X and O. |
| 746 | Creates a history entry for each recorded match result. |
| 2124 | Checks each saved round result and rebuilds score and comeback state. |
| 2136 | Replays each saved move through game rules to reconstruct the current match. |

## Generated SVG and cleanup

| LINE | WHAT THE LOOP DOES |
|---|---|
| 250 | Creates four repeated corner-circuit groups for every board cell. |
| 874 | Applies each stored attribute to a generated SVG element. |
| 879 | Cancels each scheduled surge-cleanup timer so old effects do not keep running. |

**Study tip:** The loops at lines 128-152 belong to AI decision-making. The loops at 2124-2136 rebuild a saved game from stored results and moves. When explaining them, focus on the repeated job and the data being visited.

# 2. Background, particles and electrical effects

Many visual effects are built procedurally: the code creates small elements or pixels in a loop, then positions or colors each one. These loops make the scene feel detailed without storing a separate image for every particle.

## Embers, smoke, lava and texture

| LINE | WHAT THE LOOP DOES |
|---|---|
| 316 | Creates 16 ember particles. |
| 336 | Creates 12 smoke puffs. |
| 362 | Visits canvas RGBA pixel data to add background grain. |
| 392 | Adds 30 glowing hot spots to the lava layer. |
| 411 | Creates 16 irregular dark crust plates. |
| 417 | Adds the vertices that shape each crust plate. |
| 429 | Creates nine glowing lava veins. |
| 432 | Makes six line segments for each vein. |
| 443 | Visits canvas pixel data to add fine texture grain. |
| 501 / 502 | Builds a rows-by-columns grid of pixel blocks for dissolving stamp text. |
| 508 | Shuffles those blocks so the stamp disappears in a randomized order. |

## Surges, fractures and lightning

| LINE | WHAT THE LOOP DOES |
|---|---|
| 905 | Creates two or four arms for an electrical surge. |
| 913 | Builds each surge arc from seven segments. |
| 925 | Adds three or five spark particles to each surge arm. |
| 1094 | Creates three or five fracture arms depending on the visual-effects tier. |
| 1101 | Builds each fracture arm from three line segments. |
| 1144 | Adds points and segments to the lightning-bolt path. |
| 1238 | Creates 22 sparks around the electrical impact. |

**What affects the amount?** Some loops use fixed counts, such as 16 embers. Others use a variable count, such as two or four surge arms. That lets the game vary the effect or scale its intensity.

# 3. Comeback, fire and board impacts

The comeback sequence combines pixel lettering, heat simulation, cracks, sparks and debris. Several loops process a two-dimensional canvas, so one loop handles rows while another handles columns or pixels.

## Epic comeback and collapse

| LINE | WHAT THE LOOP DOES |
|---|---|
| 1305 | Renders the two message phrases: EPIC COMEBACK and BY X/O. |
| 1308 | Processes each character in a phrase into pixel glyphs. |
| 1337 | Places active heat vents along the bottom edge. |
| 1344 | Scans horizontal positions to find where the rising flame front should appear. |
| 1346 | Writes heat values through vertical strips from each flame tip down ten pixels. |
| 1352 | Adds eight temporary radial heat sources. |
| 1359 | Uses two nested loops to diffuse and cool heat across every pixel of a 128-by-128 canvas. |
| 1370 | Creates 12 rising sparks. |
| 1388 | Builds cracks from four line segments. |
| 1397 | Adds secondary cracks from up to five cells that already have cracks. |
| 1485 | Creates 14 dust particles for the aftermath. |

## Wires, move impacts and debris

| LINE | WHAT THE LOOP DOES |
|---|---|
| 1499 | Builds the wiring group for both X and O. |
| 1501 | Draws three path layers for each player wire: housing, edge and energy. |
| 1524 | Calculates mirrored wire paths for X and O. |
| 1545 | Creates four bands or slices for the retiring-mark effect. |
| 1549 | Creates eight fragment bits for the retiring mark. |
| 1566 | Fills an audio noise buffer with random values for synthetic sound. |
| 1630 | Creates eight or 12 landing sparks, dust pieces and chips depending on impact strength. |
| 1646 | Creates 30 debris pieces for a board collapse. |
| 1678 | Creates 16 debris pieces when the replacement board lands. |
| 1717 | Creates an electrical arc to each of the nine board cells. |
| 1721 | Builds each board arc from six line segments. |

**Nested loop example - line 1359:** The outer loop moves through rows and the inner loop moves through columns. Together they visit each pixel in the heat map, update its heat, and let the flame animation rise and cool.

# 4. Circuit background and defense review

These remaining loops draw the ambient circuit field, including routes, grid lines, pulses and chip pins.

## Circuit backdrop

| LINE | WHAT THE LOOP DOES |
|---|---|
| `1891` | Draws successive points and connecting segments in one circuit bus. |
| `1970` | Creates chip boxes based on the current screen size. |
| `1984` | Creates circuit bus paths across the screen. |
| `1996` | Builds each bus from four to seven segments. |
| `2030` | Adds one or two moving pulses to each main bus. |
| `2044` | Draws vertical background grid lines. |
| `2050` | Draws horizontal background grid lines. |
| `2068` | Adds pins around each chip. |

## Quick defense answers

**What is a for loop?** It repeats instructions while a condition remains true. A counter commonly tracks which item is being handled.

**Where does Cyber Grid use loops?** In rule checks, AI search, score and save reconstruction, plus repeated visual work such as particles, textures, lightning and circuit animation.

**Which class handles the score?** ScoreBoard. BusLine and CircuitPulse handle decorative circuit visuals.

**Why are there nested loops at line 1359?** A heat map has rows and columns. The two loops visit every pixel so heat can spread and cool over time.

**Does 61 mean the project has only 61 loops?** It means 61 explicit `for (...)` statements in the main `script.js`. Array methods such as forEach, map and filter also repeat work, but they are a different syntax and are not part of this count.

The project also has server JavaScript files, but those files do not declare explicit JavaScript classes or `for (...)` loops. The CSS uses class selectors for styling; those are not JavaScript class declarations.

## Final recall check

Without looking, explain what ScoreBoard does, describe the three parts of a typical for loop, name one loop used by the game rules, and name one loop used by a visual effect. Then check the line references in the game script.